Anish Ghosh
2314909
Sem-5
B.Sc(H)C.S

Software Engineering

DATE 16.11.26
PAGE

Assignment - 1

* Section-A

1. i) Umbrella activities in software engineering are a set of ongoing, supporting tasks that span the entire software development lifecycle. They are not tied to a specific phase but are performed continuously or iteratively throughout the project to ensure quality, manage risks and support development.

No, umbrella activities are not applied evenly across the software development life cycle

- Design and Testing phases involve high complexity and ambiguity → more formal reviews, risk analysis and quality assurance.
- Deployment is short and focused → fewer reviews, more configuration finalization.
- Early phases (requirements, design) have higher uncertainty → intensive risk management.
- Late phases (maintenance) require more software configuration management and quality assurance.

Scanned with OKEN Scanner

ii) Timeline Chart is a graphical representation of a project schedule that shows tasks on a horizontal time scale.

It displays :-

a) When each task starts
b) When it ends
c) How long it takes
d) Which tasks overlap

It shows task dependencies and overlaps, is useful for monitoring deadlines and critical tasks and helps plan the sequence of tasks, track project progress against planned schedule, identify delays and adjust timelines, etc.

iii) Incremental Process Model is a software development model where the system is built in small increments (modules). Each increment adds new functionality to the previous version.

It is delivered in multiple builds, each adding features, combines elements of waterfall (each increment follows analysis → design → code → test) and iterative. Customers get working software early.

Scanned with OKEN Scanner

Example:-
For a Library Management System :-
Inc 1: Basic book search and user login
Inc 2: Book issue / return module
Inc 3: Fine calculation, report generation
Inc 4: Admin Dashboard

Each build is delivered one after another, and the customer can start using early parts of the system before the whole system is complete.

iv) 3 Agile Principles:-

a) Customer collaboration over contract negotiation - Agile focuses on working closely with customers and adapting to their feedback instead of strictly following a fixed contract.

b) Welcome changing requirements - Agile allows and encourages changes, even late in development, so the final product meets customer needs better.

c) Deliver working software frequently - Agile encourages short development cycles and frequent delivery, ensuring continuous improvement and fast feedback.

Scanned with OKEN Scanner

v) a) Functional Requirements :- These specify what the system should do - the actual functions or features.

Example :-

- The system must allow users to log in using a username and password.
- The ATM must allow cash withdrawal.

b) Non-Functional Requirements :- These specify how the system should behave - quality attributes and constraints.

Example :-

- The system must respond within 2 seconds. (Performance)
- The system should handle 10,000 users simultaneously (Scalability)
- The data must be encrypted (Security)

Functional requirements ensure correct features are built. Non-functional requirements ensure the software is usable, fast, secure and reliable.

vi) a) High Cohesion :-

- Cohesion means how strongly related the functions inside a module are.
- Module performs one well-defined task.
- Easier to maintain, understand and reuse.

Scanned with OKEN Scanner

b) Low Coupling :-

- Coupling means interdependence between modules.
- Modules are independent, changes in one module do not break others.

Designing with high cohesion and low coupling improves modularity, maintainability, reusability, testing and debugging, flexibility for future changes, etc.

vii) • Importance of Testing :-

a) Ensures software meets requirements and quality standards.
b) Detects defects early, reducing cost of fixing
c) Improves reliability, performance and security.
d) Builds confidence for users and clients
e) Prevents failures after deployment

- Software can still fail after acceptance testing due to :-

a) Uncovered scenarios - Acceptance tests may not cover all real-world use cases.
b) Environment differences - Production environment may differ from test environment.
c) Human errors / operational mistakes - Users may use the software in unexpected ways.
d) Integration with other systems - Real systems interact with hardware, networks or databases that may behave differently.

Scanned with OKEN Scanner

e) Updates and patches - Later modifications may introduce new bugs.

viii) • Impact of Risk refers to the loss or damage the the project will face if the risk occurs, usually assessed as High/Medium/Low or a numeric value representing cost of loss.

• Formula - Risk Exposure (RE):

Risk Exposure (RE) = Probability of Risk × Impact of Risk

Probability of Risk → Likelihood risk will occur

Impact of Risk → Cost/Damage if it happens

ix) CMMI (Capability Maturity Model Integration) has 5 capability levels:-

• Level 1 (Initial) - Processes are chaotic and unpredictable
• Level 2 (Managed) - Basic project management processes exist. Planning, tracking and requirements management are followed.
• Level 3 (Defined) - Organization-wide standardized ~~process~~

processes are used. Documentation and process improvement are formal.

- Level 4 (Quantitatively Managed) - Processes are measured and controlled using quantitative data.
- Level 5 (Optimizing) - Continuous process improvement using feedback, innovation and defect prevention.

Section - B

2. Cyclomatic Complexity measures the number of linearly independent paths through a program's control-flow graph.

- It gives a lower ~~bound~~ bound on the number of test cases needed for full branch coverage
- It highlights modules that are complex and likely to contain defects
- helps plan testing effort and guides refactoring (high CC → candidate for simplification)

* Ways to compute :-

i) Edge-Node Formula :-

$V(G) = E - N + 2P$ → No. of connected components

E → Edges, N → Nodes

ii) Decision points Method

$V(G) = P + 1$

↓

No. of predicate / decision nodes (Nodes with more than one outgoing edge)

iii) Regions / Faces Method (planar embedding)

$V(G) = R$

↓

No. of independent regions (including exterior) in a planar drawing of the graph.

* $P = 4$

$V(G) = P + 1 = 5$

~~List of~~ 5 independent paths :-

i) A → B → C → D → G → H → I → J

ii) A → B → C → D → G → H → J

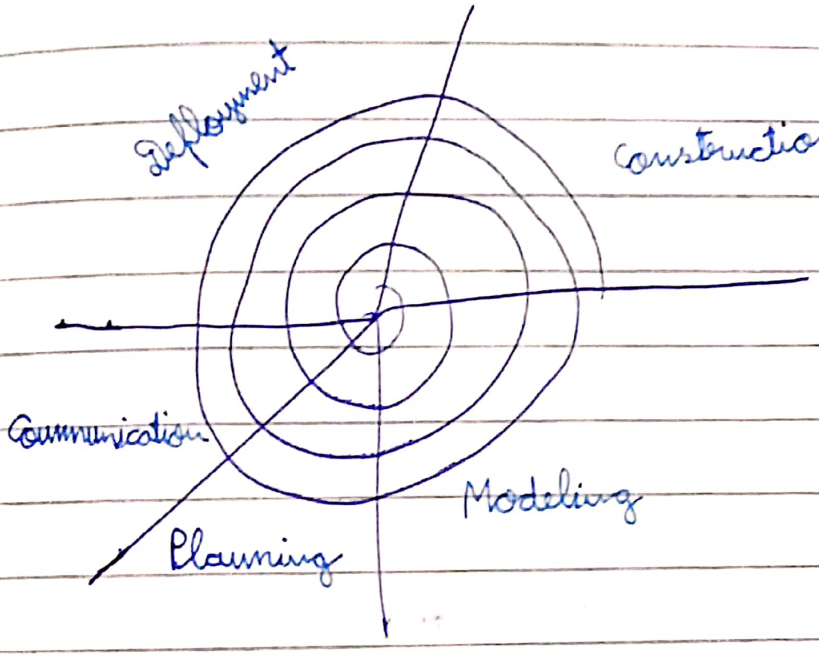
iii) A → B → E → F → G → H → I → J

iv) A → B → E → D → G → H → I → J

v) A → B → C → B → E → F → G → H → I → J

Scanned with OKEN Scanner

3. a) The spiral model is a risk-driven software development process that combines the iterative nature of the incremental model with the structure of the waterfall model. It is used for large, complex and high-risk projects, where each loop / phase of the spiral repeats the development process and includes a focus on risk analysis and management, leading to continuous refinement and a flexible approach to evolving requirements.



- Risk affects the model in the following ways:-

i) Determines the activities of each loop
More risk → More extensive risk analysis, prototyping and verification
Low risk → Smaller loops, faster progress.

ii) Decides the no. of iterations - High-risk projects require more loops to reduce uncertainty.

iii) Affects cost and time - Risk handling increases early cost but reduces failure probability later.

Scanned with OKEN Scanner

iv) Determines whether project continues or stops - If risks cannot be controlled, development may be terminated early, saving money.

v) Shapes the ~~environment path~~ development path - Different risks guide different routes.

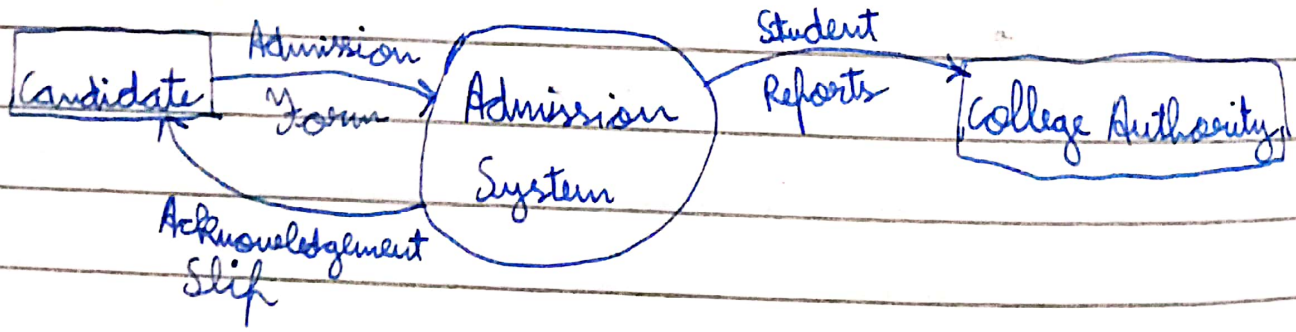
(b) Agile processes are built on the assumption that :-

i) Requirements will change frequently - Customers may not know requirements fully at the start, and changes must be welcomed even late in development.

ii) Small, incremental deliveries are better than big releases - Working software delivered quickly provides faster customer feedback and reduces risk.

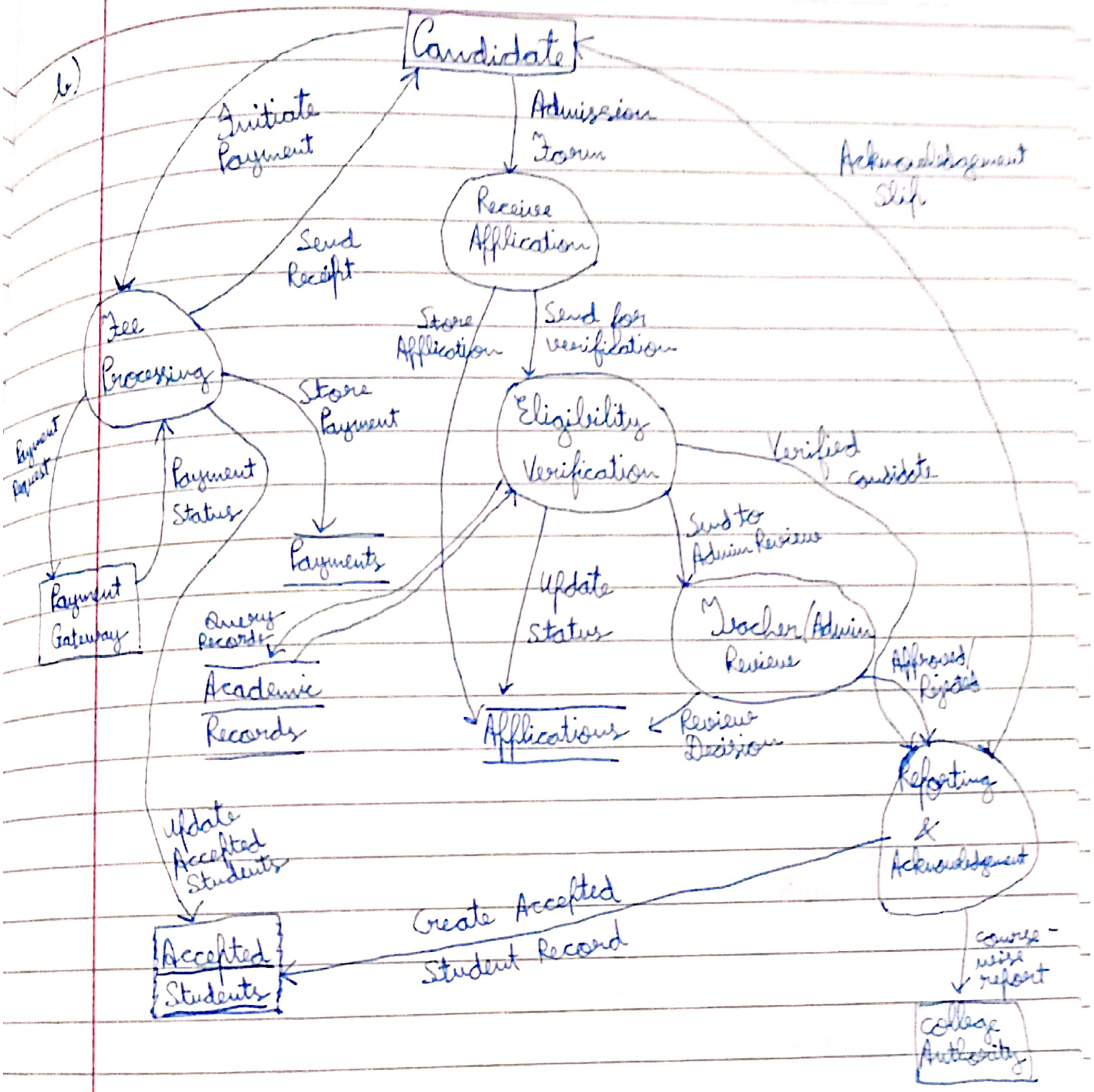
iii) People and communication matter more than processes - Motivated individuals, close collaboration and rapid feedback are essential for success.

4. a)



Scanned with OKEN Scanner

b)



c) i) P1: Applications

Application ID = digit (6)

Candidate ID = digit (6)

Name = alphabet (30)

Scanned with OKEN Scanner

DOB = digit(2) + / + digit(2) + / + digit(4)
Address = alphanum(60)
Contact = digit(10)
Course Applied = alphabet(10)
Marks = digit(3)
Documents = alphanum(20)
Status = {Submitted | Under Verification | Verified | Rejected}
Verification Notes = alphabet(50)

ii) D2: Academic Records PB
Candidate ID = digit(6)
Institution Name = alphabet(40)
Exam Roll = alphanum(10)
Year = digit(4)
Subject Marks = digit(3)
Verification Status = {Matched | Not Matched}
Verified on = digit(2) + / + digit(2) + / + digit(4)

iii) D3: Accepted Students
~~Accepted Students~~
Accepted ID = digit(6)
Candidate ID = digit(6)
Course ID = alphabet(10)
Admission Date = digit(2) + / + digit(2) + / + digit(4)
Fee Status = {Pending | Paid}

Scanned with OKEN Scanner

Seat Number = alphanum (5)
Batch = alphaleet (5)

ii) D4 : Payments
Payment ID = digit (6)
Candidate ID = digit (6)
Application ID = digit (6)
Amount = digit (5)
Transaction ID = alphanum (15)
Payment Status = {Success | Failed | Pending}
Timestamp = digit (2) + / + digit (2) + / + digit (4) +
digit (2) + : digit (2)

5. a)

Low WT		Avg WT		High WT	Sum
4 x 3	+	4 x 4	+	4 x 6	52
10 x 4	+	10 x 5	+	10 x 7	160
2 x 3	+	2 x 4	+	2 x 6	26
3 x 7	+	3 x 10	+	3 x 15	96
1 x 5	+	1 x 7	+	1 x 10	22

⇒ 356
(Count Total)

FP = Count Total x Adjustment Factor
= 356 x 1.5
⇒ 534 (Ans)

Scanned with OKEN Scanner

b) PRE for analysis phase :-

$$PRE = \frac{\text{Defects found in that phase}}{\text{Defects found in that phase + Defects found later that originated in that phase}}$$

$$\Rightarrow \frac{20}{20+10} = \frac{20}{30} = 0.6667 = 66.67\%$$

6. a) i) Quality Planning - Defines quality standards, procedures, techniques and metrics that will be used during development.

ii) Process Definition and Implementation - Establishing documented development processes and ensuring the team follows them consistently.

iii) Reviews & Inspections - Conducting formal technical reviews, walkthroughs and inspections to detect defects early.

iv) Training and Skill Development - Providing training for developers and testers to ensure they understand tools, techniques, standards.

v) Test planning and Quality Measurement - Creating test strategies, maintaining quality metrics and deciding how quality will be evaluated.

b) Characteristics of Risk :-

i) Uncertainty - Risk may or may not occur.

Scanned with OKEN Scanner

ii) Loss - If it occurs, it causes negative consequences
iii) Probability - Each risk has a chance of occurence
iv) Impact - Amount of damage the risk can cause
v) Time Frame - Risks have a time window during which they are relevant.

* Types of Risks:-

i) Project Risks - Affect project schedule, resources, people like :-
- Team turnover
- Unrealistic Schedules
- Budget overrun
- Communication Issues

ii) Technical Risks - Affect quality and performance of the software like :-
- Technology may not work as expected
- Integration issues
- Performance bottlenecks
- New tools not understood

iii) Business Risks - Affect the product's long-term value like :-
- Product may not be accepted by market
- Competitor products
- Change in customer needs
- Loss of funding

Scanned with OKEN Scanner

iv) Operational Risks - Related to environment or infrastructure like :-
- Server failure
- Network outage
- Deployment issues

v) External Risks - Outside the control of the project team like :-
- Legal changes
- Regulatory compliance
- Cybersecurity attacks

7. i) a) ~~Coupling :-~~ Cohesion :-
- Measures how strongly the elements inside a module are related
- High cohesion is desireable

b) Coupling :-
- Measures how strongly one module depends on other modules
- Low coupling is desirable

ii) a) Verification :-
- Checks whether the software follows specifications
- Done without executing the code

Scanned with OKEN Scanner

b) Validation :-
- Checks whether the software meets user needs
- Done by executing the code.

iii) a) Reactive Risk Strategy :-
- Acts after a risk has ~~occurred~~ occurred
- Focuses on damage control and recovery

b) Proactive Risk Strategy :-
- Identifies and handles risks before they occur
- Focuses on prevention and minimization

iv) a) Milestone :-
- A checkpoint or event that marks progress in a project
- Indicates completion of a phase or activity

b) Deliverable :-
- A tangible output or work product produced
- Given to customer or stakeholders.

v) a) Alpha Testing :-
- Performed by internal team
- Conducted in a controlled environment

Scanned with OKEN Scanner

b) Beta Testing :-

- Performed by external users/customers
- Conducted in real-world environment

8. a) A Software Requirements Specification (SRS) is a formal document that describes what the software must do. It includes functional requirements, non-functional requirements, constraints, interfaces, performance and reliability needs

- SRS bridges communication gap by :-

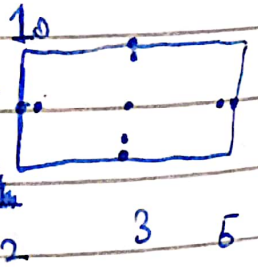
i) Acting as a contract between customer and developer
ii) Providing a common vocabulary
iii) Avoiding misunderstandings
iv) Improving communication through structured documentation
v) Helping in validation

b) Boundary Value Test-Cases for $f(a, b)$

$2 \le a \le 5$

$1 \le b \le 10$

$4n+1 = 9$ cases



a	b
2	5
3	5
3	1
3	2
3	5

3	9
3	10
4	5
5	5

Scanned with OKEN Scanner